EJERCICIO1

```
public static int signo(int x) {
if (x > 0) {
return 1;
} else if (x < 0) {
return -1;
}
return 0;
}
```

1. Calcula la complejidad ciclomática.

La complejidad ciclomatica seria 3,ya que esta la de mayor que 0, la de menor que 0, y la restante que seria el 0

2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.

Un valor negativo, uno positivo y un cero

3. Implementa la clase SignoTest con JUnit.

ID	DESCRIPCION	ENTRADA	ESPERADO	ESTADO
01	Positivo	3	1	if (x > 0) { return 1;
02	Negativo	-3	-1	else if (x < 0) { return -1;
03	Neutro	0	0	} return 0;

EJERCICIO 2

Metodo calcular edad

```java
package com.MiguelCampos.tema4gradle;

public class Edad {  no usages  new *
    public static String clasificarEdad(int edad) {  no usages  new *
        if (edad < 0) {
            throw new IllegalArgumentException("Edad no válida");
        }
        if (edad < 6) {
            return "Infancia";
        } else if (edad < 12) {
            return "Niñez";
        } else if (edad < 18) {
            return "Adolescencia";
        } else if (edad < 25) {
            return "Juventud";
        } else if (edad < 60) {
            return "Adultez";
        } else {
            return "Vejez";
        }
    }
```

1. Calcula la complejidad ciclomática.

6+1

2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.

7 casos como minimo

ID	Descripción	Entrada	Esperado	Estado
01	Edad negativa	-1	Error	`if (edad < 0)`
02	Infancia	5	"Infancia"	`edad < 6`
03	Niñez	10	"Niñez"	`edad < 12`

04	Adolescencia	15	"Adolescencia"	`edad < 18`
05	Juventud	20	"Juventud"	`edad < 25`
06	Adultez	40	"Adultez"	`edad < 60`
07	Vejez	70	"Vejez"	`else`

3. Implementa ClasificarEdadTest.

```java
import static org.junit.jupiter.api.Assertions.*;

import com.MiguelCampos.tema4gradle.Edad;
import org.junit.jupiter.api.Test;

class EdadUtilsTest {  new *

    @Test  new *
    void testEdadNegativa() {
        assertThrows(IllegalArgumentException.class, () -> {
            Edad.clasificarEdad(-1);
        });
    }

    @Test  new *
    void testCategoriasPrincipales() {

        assertEquals( expected: "Infancia", Edad.clasificarEdad(2));
        assertEquals( expected: "Niñez", Edad.clasificarEdad(9));
        assertEquals( expected: "Adolescencia", Edad.clasificarEdad(16));
        assertEquals( expected: "Juventud", Edad.clasificarEdad(20));
        assertEquals( expected: "Adultez", Edad.clasificarEdad(40));
        assertEquals( expected: "Vejez", Edad.clasificarEdad(70));
    }

    @Test  new *
    void testCasosLimite() {

        assertEquals( expected: "Infancia", Edad.clasificarEdad(0));
        assertEquals( expected: "Adultez", Edad.clasificarEdad(59));
    }
}
```

Ejercicio 3. Método contarPositivos

Se proporciona el siguiente método:

```java
public class ContarPositivos{  no usages
        public static int contarPositivos(int[] datos) {
            int contador = 0;
            for (int i = 0; i < datos.length; i++) {
                if (datos[i] > 0) {
                    contador++;
                }
            }
            return contador;
        }
    }
```

1. Calcula la complejidad ciclomática.

2+1

2. Diseña pruebas para cubrir: • array vacío, • array con todos negativos, • array con mezcla de positivos, negativos y ceros.

ID	Descripción	Entrada	Esperado	Rama cubierta
01	Array vacío	`{}`	0	No entra en el bucle `for`.
02	Todo negativos	`{-1, -5}`	0	Entra en el `for`, pero no se cumple el `if`.
03	Mezcla total	`{1, -2, 0, 3}`	2	Cubre tanto la subida del contador como el del `if`.

3. Implementa ContarPositivosTest.

```java
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class ArrayTest {  new *
    @Test  new *
    void testArrayVacio() {
        int[] datos = {};
        assertEquals( expected: 0, ArrayUtils.contarPositivos(datos),  message: "Un array vacío debe devolver 0");
    }

    @Test  new *
    void testArrayNegativo() {
        int[] datos = {-1, -10, -5};
        assertEquals( expected: 0, ArrayUtils.contarPositivos(datos),  message: "Si todos son negativos el resultado es 0");
    }

    @Test  new *
    void testArrayMixto() {

        int[] datos = {1, -2, 0, 5, -10, 10};
        assertEquals( expected: 3, ArrayUtils.contarPositivos(datos),  message: "Tiene que contar 3 positivos (1, 5, 10)");
    }
}
```

Ejercicio 4. Método calificacion

```java
public class Calificacion {  no usages  new *
    public static String calificacion(int nota) {  no usages  new *
        if (nota < 0 || nota > 10) {
            throw new IllegalArgumentException("Nota no válida");
        }
        return switch (nota) {
            case 0, 1, 2, 3, 4 -> "SUSPENSO";
            case 5 -> "SUFICIENTE";
            case 6 -> "BIEN";
            case 7, 8 -> "NOTABLE";
            default -> "SOBRESALIENTE";
        };
    }

}
```

1. Calcula la complejidad ciclomática

6+1

2. Diseña los casos mínimos para cubrir todas las alternativas del switch y la validación inicial.

7 casos

3. Implementa CalificacionTest.

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

public class TestCalificacion {  new *
    @ParameterizedTest  new *
    @ValueSource(ints = {0, 2, 4})
    void testSuspensos(int nota) {
        assertEquals( expected: "SUSPENSO", Calificacion.calificacion(nota));
    }
    @Test  new *
    void testNotasNoValidas() {
        assertThrows(IllegalArgumentException.class, () -> Calificacion.calificacion( nota: -1));
        assertThrows(IllegalArgumentException.class, () -> Calificacion.calificacion( nota: 11));
    }
    @Test  new *
    void testNotas() {
        assertAll(
                () -> assertEquals( expected: "SUFICIENTE", Calificacion.calificacion( nota: 5)),
                () -> assertEquals( expected: "BIEN", Calificacion.calificacion( nota: 6)),
                () -> assertEquals( expected: "NOTABLE", Calificacion.calificacion( nota: 7)),
                () -> assertEquals( expected: "SOBRESALIENTE", Calificacion.calificacion( nota: 10))
        );
    }
}
```

Ejercicio 5. Crear un método esBisiesto

Implementa un método con la siguiente firma:

public static boolean esBisiesto(int anyo)

```java
public class Bisiesto {  no usages  new *
        public static boolean esBisiesto(int anyo) {  no usages  new *
            if ((anyo % 4 == 0 && anyo % 100 != 0) || (anyo % 400 == 0)) {
                return true;
            } else {
                return false;
            }
        }
    }
```

1. Calcula su complejidad ciclomática a partir de tu código.

3+1

2. Diseña los casos de prueba necesarios para cubrir todas las decisiones.

ID	Descripción	Entrada	Esperado	Rama
01	Divisible por 4 y no por 100	2024	`true`	Cumple la primera parte del `if`
02	Divisible por 100 pero no por 400	1900	`false`	Falla la primera y segunda parte
03	Divisible por 400	2000	`true`	Cumple la segunda parte
04	No divisible por 4	2023	`false`	No cumple ninguna parte

3. Implementa EsBisiestoTest.

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
public class BisiestoTest {   new *
    @Test   new *
        void testEsBisiesto() {
            assertAll(
                    () -> assertEquals(Bisiesto.esBisiesto( anyo: 2024),  actual: true,  message: "2024 es bisiesto"),
                    () -> assertFalse(Bisiesto.esBisiesto( anyo: 1900),  message: "1900 No es bisiesto"),
                    () -> assertTrue(Bisiesto.esBisiesto( anyo: 2000),  message: "2000 es bisiesto"),
                    () -> assertFalse(Bisiesto.esBisiesto( anyo: 2023),  message: "2023 No es bisiesto")
            );
        }
    }
```

Ejercicio 6. Implementar Piedra – Papel – Tijera

Implementa el método:

public static String jugar(String jugador1, String jugador2)

```java
public class Jugar {  no usages  new *
        public static String jugar(String jugador1, String jugador2) {  no usages  new *
            String j1 = (jugador1 != null) ? jugador1.toUpperCase() : "";
            String j2 = (jugador2 != null) ? jugador2.toUpperCase() : "";
            if (!esValida(j1) || !esValida(j2)) {
                throw new IllegalArgumentException("Jugada no válida");
            }
            if (j1.equals(j2)) {
                return "EMPATE";
            }

            if ((j1.equals("PIEDRA") && j2.equals("TIJERA")) ||
                    (j1.equals("TIJERA") && j2.equals("PAPEL")) ||
                    (j1.equals("PAPEL") && j2.equals("PIEDRA"))) {
                return "GANADOR JUGADOR 1";
            } else {
                return "GANADOR JUGADOR 2";
            }
        }

        private static boolean esValida(String jugada) {  2 usages  new *
            return jugada.equals("PIEDRA") || jugada.equals("PAPEL") || jugada.equals("TIJERA");
        }
    }
```

1. Calcular la complejidad ciclomática.

6+1

2. Diseñar los tests mínimos para cubrir todas las ramas.

ID	Descripción	Entrada J1 / J2	Esperado	Rama
01	Jugador 1 inexistente	"FUEGO", "PIEDRA"	Excepcion	Validación de jugada 1
02	Jugador 2 inexistente	"PIEDRA", "SPOCK"	Excepcion	Validación de jugada 2
03	Empate	"PAPEL", "PAPEL"	EMPATE	Empate

04	Gana jugador1 (Piedra)	"PIEDRA", "TIJERA"	GANADOR JUGADOR 1	Piedra gana a tijera
05	Gana jugador 1 (Tijera)	"TIJERA", "PAPEL"	GANADOR JUGADOR 1	Tijera gana a papel
06	Gana jugador 1 (Papel)	"PAPEL", "PIEDRA"	GANADOR JUGADOR 1	Papel gana a piedra
07	Gana jugador 2	"TIJERA", "PIEDRA"	GANADOR JUGADOR 2	Tijera pierde con piedra

3. Implementar PiedraPapelTijeraTest.

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

    class PiedraPapelTijeraTest {  new *

        @Test  new *
        void testJugadasErroneas() {
            assertThrows(IllegalArgumentException.class, () -> Jugar.jugar("PIEDRA", "HOLA"));
        }
        @Test  new *
        void testEmpate() {
            assertEquals( expected: "EMPATE", Jugar.jugar("PIEDRA", "PIEDRA"));
        }

        @Test  new *
        void testVictoriasJugador1() {
            assertAll(
                    () -> assertEquals( expected: "GANA JUGADOR 1", Jugar.jugar("PIEDRA", "TIJERA")),
                    () -> assertEquals( expected: "GANA JUGADOR 1", Jugar.jugar("TIJERA", "PAPEL")),
                    () -> assertEquals( expected: "GANA JUGADOR 1", Jugar.jugar("PAPEL", "PIEDRA"))
            );
        }

        @Test  new *
        void testVictoriaJugador2() {
            assertEquals( expected: "GANA JUGADOR 2", Jugar.jugar("TIJERA", "PIEDRA"));
        }
    }
```

Ejercicio 7.

Implementar Validador de contraseñas

Implementa el método: public static int evaluarPassword(String password)

2. Calcular complejidad ciclomática.

5+1

3. Diseñar casos de prueba representativos.

ID	Descripción	Entrada	Esperado (Puntos / Categoria)	Rama
01	Pocos numeros	"111"	1 / "DEBIL"	Solo cumple regla de números
02	Letras	"miguel"	2 / "DEBIL"	Letras y minúsculas
03	Letras y numeros	"Miguel1234"	4 / "MEDIA"	Larga, Mayuscula, Minusculas y numeros
04	Con todos los criterios	"P@rg7ds13"	5 / "FUERTE"	Cumple los 5 criterios
05	Caso nulo	null	0 / "DEBIL"	Control de errores

4. Implementar PasswordTest.

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class PasswordTest {  new *

    @Test  new *
    void testPocosNumeros() {
        assertEquals( expected: 1, Validador.evaluarPassword("111"));
    }

    @Test  new *
    void testLetras() {
        assertEquals( expected: 2, Validador.evaluarPassword("miguel"));
    }

    @Test  new *
    void testLetrasYNumeros() {
        assertEquals( expected: 4, Validador.evaluarPassword("Miguel1234"));
    }

    @Test  new *
    void testConTodosLosCriterios() {
        assertEquals( expected: 5, Validador.evaluarPassword("P@rg7ds13"));
    }

    @Test  new *
    void testCasoNulo() {
        assertEquals( expected: 0, Validador.evaluarPassword(null));
    }
}
```